Học Sâu (Deep Learning)

Chương 11: Phân vùng Hình ảnh (Image Segmentation)

Đại học Đông Á

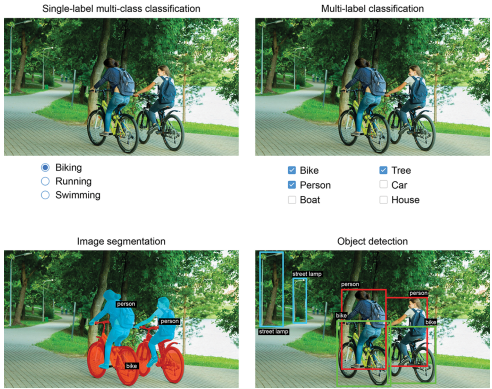
August 14, 2026

Nội dung chương

1. 1. Tổng quan các Bài toán Thị giác Máy tính
2. 2. Xây dựng Mô hình Phân vùng Semantic
3. 3. Siêu mô hình Nền tảng: Segment Anything (SAM)
4. 4. Thực hành Kỹ nghệ Prompting trên SAM
5. Tổng kết

Ba nhánh chính của Thị giác Máy tính

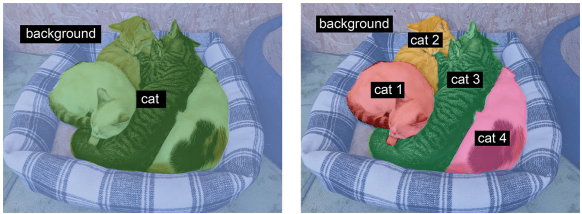
- **Image Classification (Phân loại):** Gán nhãn cho toàn bộ bức ảnh ("Ảnh này có con mèo").
- **Object Detection (Phát hiện vật thể):** Vẽ hộp giới hạn (Bounding box) quanh vật thể và phân loại chúng ("Con mèo ở tọa độ x, y").
- **Image Segmentation (Phân vùng):** Phân loại ở cấp độ **Điểm ảnh (Pixel)**. Cắt chính xác đường viền của vật thể ("Những pixel nào thuộc về con mèo?").



Hình 11.1: Ba bài toán cốt lõi trong CV

Phân loại Bài toán Image Segmentation

- **Semantic Segmentation:** Phân loại pixel theo "Danh mục" (Category). Nếu có 2 con mèo, cả 2 sẽ có cùng màu (cùng nhãn "Mèo").
- **Instance Segmentation:** Phân loại pixel theo "Cá thể" (Instance). Phân biệt rạch ròi "Con mèo 1" và "Con mèo 2".
- **Panoptic Segmentation:** Sự kết hợp cao cấp của cả Semantic và Instance Segmentation.



Hình 11.2: Semantic vs Instance Segmentation

Bài toán: Phân tách Tiền cảnh và Hậu cảnh

- Ta sẽ xây dựng một mô hình CNN từ đầu để thực hiện Semantic Segmentation trên tập dữ liệu Oxford-IIIT Pets (Chó & Mèo).
- **Mục tiêu:** Phân biệt đâu là chủ thể (Vật nuôi), đâu là đường viền, và đâu là hậu cảnh (Background).
- Ảnh đầu vào: $200 \times 200 \times 3$.



Hình 11.3: Ảnh đầu vào

Tải Dữ liệu Oxford-IIIT Pets

Sử dụng các lệnh `wget` và `tar` trên Colab để tải hình ảnh (images) và nhãn mặt nạ (annotations).

```
!wget http://www.robots.ox.ac.uk/~vgg/data/pets/data/images.tar.gz
!wget http://www.robots.ox.ac.uk/~vgg/data/pets/data/annotations.tar.gz
!tar -xf images.tar.gz
!tar -xf annotations.tar.gz
```

Sắp xếp danh sách file (Xóa các file ẩn rác):

```
import pathlib

input_dir = pathlib.Path("images")
target_dir = pathlib.Path("annotations/trimaps")

input_img_paths = sorted(input_dir.glob("*.jpg"))
# Ignores some spurious files in the trimaps directory that start with "."
target_paths = sorted(target_dir.glob("[!.]*.png"))
```

Khái niệm Mặt nạ (Mask)

- Dữ liệu nhãn (Ground Truth) trong bài toán này không phải là một con số, mà là một **Mặt nạ (Mask)**.
- Kích thước Mask tương đương ảnh gốc ($200 \times 200 \times 1$).
- Mỗi pixel mang 1 trong 3 giá trị:
  1. Foreground (Tiền cảnh)
  2. Background (Hậu cảnh)
  3. Contour (Đường viền)



Hình 11.4: Mặt nạ phân vùng (Ground truth)

Hàm Load Ảnh và Mặt nạ

Định nghĩa hàm đọc file từ ổ đĩa và chuyển thành Numpy array. Mặt nạ được xử lý thành thang xám (grayscale) và trừ 1 để đưa các nhãn về $[0, 1, 2]$.

```
def path_to_input_image(path):
    return img_to_array(load_img(path, target_size=img_size))

def path_to_target(path):
    img = img_to_array(
        load_img(path, target_size=img_size, color_mode="grayscale")
    )
    # Subtracts 1 so that our labels become 0, 1, and 2
    img = img.astype("uint8") - 1
    return img
```

Nạp Hình ảnh vào mảng NumPy

Khởi tạo hai mảng kích thước cực lớn chứa toàn bộ Data. Ảnh có 3 kênh màu, Mặt nạ chỉ có 1 kênh số nguyên.

```
import numpy as np
import random
from keras.utils import load_img, img_to_array

img_size = (200, 200)
num_imgs = len(input_img_paths)

# Shuffles the file paths
random.Random(1337).shuffle(input_img_paths)
random.Random(1337).shuffle(target_paths)

input_imgs = np.zeros((num_imgs,) + img_size + (3,), dtype="float32")
targets = np.zeros((num_imgs,) + img_size + (1,), dtype="uint8")

# Loops over all elements to fill numpy arrays...
```

Chia tập Huấn luyện và Xác thực

Tương tự bài toán phân loại ảnh, ta cần chia dữ liệu để tránh Overfitting.

```
# Reserves 1,000 samples for validation
num_val_samples = 1000

# Splits the data into a training and a validation set
train_input_imgs = input_imgs[:-num_val_samples]
train_targets = targets[:-num_val_samples]

val_input_imgs = input_imgs[-num_val_samples:]
val_targets = targets[-num_val_samples:]
```

Dù kích thước tập dữ liệu tương đối nhỏ (hơn 7000 ảnh), nhưng nhờ kỹ thuật Data Augmentation và kiến trúc CNN mạnh, ta vẫn có thể học tốt.

Kiến trúc Encoder - Decoder (U-Net style)

Mạng nơ-ron phân vùng có hình phễu, chia làm 2 nửa:

- **Encoder (Mã hóa):** Dùng `Conv2D` với `strides=2` để **Hạ độ phân giải** ảnh xuống (Ví dụ: 200 $\rightarrow$ 100 $\rightarrow$ 50). Nhằm trích xuất thông tin khái niệm (What).
- **Decoder (Giải mã):** Dùng `Conv2DTranspose` với `strides=2` để **Tăng độ phân giải** ảnh lên lại (Ví dụ: 50 $\rightarrow$ 100 $\rightarrow$ 200). Nhằm định vị lại không gian gốc (Where).

Toán học của Transposed Convolution

- Để tăng độ phân giải (Upsampling) mà vẫn duy trì khả năng học hỏi (Learnable), ta dùng **Tích chập Chuyển vị (Transposed Convolution)**.
- Nó hoạt động ngược lại với Tích chập thông thường:
  - Nhận 1 giá trị đầu vào (Pixel).
  - Nhân giá trị đó với một Bộ lọc (Kernel).
  - Phát tán (Broadcast) kết quả ra một vùng không gian lớn hơn ở đầu ra.
- Nhờ đó, mạng có thể học cách "điền" thông tin chi tiết vào những vùng ảnh đã bị thu nhỏ trước đó.

Định nghĩa Nửa thứ nhất - Encoder

Hạ độ phân giải ảnh để nén đặc trưng. Lưu ý ta dùng `strides=2` thay cho MaxPooling để giữ lại thông tin tọa độ không gian (Location info).

```python
import keras
from keras.layers import Rescaling, Conv2D, Conv2DTranspose

def get_model(img_size, num_classes):
    inputs = keras.Input(shape=img_size + (3,))
    x = Rescaling(1.0 / 255)(inputs)

    # Encoder part: Downsampling
    x = Conv2D(64, 3, strides=2, activation="relu", padding="same")(x)
    x = Conv2D(64, 3, activation="relu", padding="same")(x)
    x = Conv2D(128, 3, strides=2, activation="relu", padding="same")(x)
    x = Conv2D(128, 3, activation="relu", padding="same")(x)
    x = Conv2D(256, 3, strides=2, padding="same", activation="relu")(x)
    x = Conv2D(256, 3, activation="relu", padding="same")(x)
```

Định nghĩa Nửa thứ hai - Decoder

Sử dụng `Conv2DTranspose` khôi phục lại độ phân giải 200×200 ban đầu. Output cuối cùng có 3 kênh (tương ứng 3 class) với Softmax.

```
    # Decoder part: Upsampling
    x = Conv2DTranspose(256, 3, activation="relu", padding="same")(x)
    x = Conv2DTranspose(256, 3, strides=2, activation="relu", padding="same")(x)
    x = Conv2DTranspose(128, 3, activation="relu", padding="same")(x)
    x = Conv2DTranspose(128, 3, strides=2, activation="relu", padding="same")(x)
    x = Conv2DTranspose(64, 3, activation="relu", padding="same")(x)
    x = Conv2DTranspose(64, 3, strides=2, activation="relu", padding="same")(x)

    # Ends with per-pixel softmax classifier
    outputs = Conv2D(num_classes, 3, activation="softmax", padding="same")(x)
    return keras.Model(inputs, outputs)

model = get_model(img_size=img_size, num_classes=3)
```

Độ đo IoU (Intersection over Union)

IoU đo phần trăm giao nhau giữa Mặt nạ thực tế và Mặt nạ dự đoán. IoU = Giao / Hợp (Nằm trong khoảng 0-1).

```
foreground_iou = keras.metrics.IoU(
    # Specifies the total number of classes
    num_classes=3,
    # Specifies the class to compute IoU for (0 = foreground)
    target_class_ids=(0,),
    name="foreground_iou",
    # Our targets are sparse (integer class IDs).
    sparse_y_true=True,
    # But our model predictions are a dense softmax!
    sparse_y_pred=False,
)
```